

CI/CD with GitHub Actions & Jenkins

Harikrishnan Balagopal · IBM Research

We look at GitHub Actions, then Jenkins, then how to make the two drive each other for complex workflows like cross-repo integration tests.

Agenda (~60 min)

1. **GitHub Actions (GHA)** : what it is, why it exists (~5 min)
2. **GHA basics** : the vocabulary and the moving parts (~12 min)
3. **A real public example** : `torch-spyre` on GitHub (~10 min)
4. **Jenkins** : what it is, when you reach for it (~5 min)
5. **Jenkins basics** : declarative pipelines, agents, parallelism (~12 min)
6. **A real internal example** : a shared Jenkins that orchestrates many repos (~8 min)
7. **Jenkins → GHA → Jenkins** : integrating both for integration tests (~8 min)
8. **References** (~2 min)

Overview

CI = Continuous Integration: every change is merged, built, and tested automatically.

CD = Continuous Deployment: every passing change ships to production.

A **CI/CD pipeline** is the automation that does this: an ordered set of stages a change flows through, where a failure at any stage stops it and reports back.

```
commit —> build —> test —> package —> release —> deploy
|
trigger      (a failure at any stage halts the pipeline and reports back)
```

- **GitHub Actions** and **Jenkins** are two engines that drive this pipeline.
- Both express "**pipeline as code**" that lives in the repo and is versioned with it.
- They are **not** mutually exclusive. Later we use **both together**.

Goal today: understand the moving parts of each, and the seam where they connect.

Part 1 : GitHub Actions

What it is and why it exists

What is GitHub Actions?

- **CI/CD built into GitHub.** No separate server to run: it is part of the repository.
- A **workflow** is a YAML file in `.github/workflows/`. When an **event** happens (a push, a pull request, a schedule), GitHub runs the workflow.
- Work runs on a **runner**: a machine, either **GitHub-hosted** (managed, ephemeral) or **self-hosted** (your own, for special hardware such as GPUs).
- Huge **Marketplace** of reusable "Actions" you drop into a step with `uses:`.

Why it exists: to make "every change is built and tested automatically" the default, with zero infrastructure to set up. It is the fastest way to add CI to a GitHub repo.

Part 2 : GitHub Actions basics

The vocabulary and the moving parts

The vocabulary

- **Workflow** : one automated process, a YAML file in `.github/workflows/`.
- **Event** : what triggers it (`on:` `[push, pull_request, workflow_dispatch, schedule]`).
- **Job** : a set of steps that run together on one runner.
- **Step** : a single command (`run:`) or a reusable **Action** (`uses:`).
- **Action** : a packaged, reusable step you call with `uses:` (from the Marketplace or your own repo).
- **Runner** : the machine a job runs on (`runs-on:`).
- **Matrix** : run the same job across many parameter combinations.
- **Artifact** : a file passed between jobs or kept after the run (build output, test report).

Jobs run in parallel by default; use `needs:` to make one job wait for another.

A minimal workflow

```
name: CI
on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # a Marketplace Action
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -e .              # a shell command
      - run: pytest -q
```

`on:` = trigger · `jobs:` = the belt · `steps:` = the work · `uses:` = reuse · `run:` = a command.

Matrix : one job, many combinations

```
jobs:
  test:
    strategy:
      fail-fast: false          # let all combos finish, see every failure
    matrix:
      python: ['3.10', '3.11', '3.12']
      os: [ubuntu-latest, macos-latest]
    runs-on: ${matrix.os}
    steps:
      - run: pytest              # runs 3 × 2 = 6 times
```

- One definition expands into many parallel jobs.
- The axes can be anything: OS, versions, and (in ML) models, datasets, or hardware types.

Reuse : do not copy-paste pipelines

Composite action

- Lives in `.github/actions/...`.
- Bundles several steps into one `uses:`.
- Good for "the same 5 setup steps everywhere".

Reusable workflow

- A whole job graph, `on: workflow_call`.
- Called by other workflows with **inputs**.
- Good for "one CI engine, many thin callers".

Gotcha: a reusable workflow's *internal* job names are not top-level status checks.

So each caller adds its **own fan-in "gate" job**, and the branch protection rule references that stable gate name.

Caching and self-hosted runners

Caching

```
- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: pip-${{ hashFiles('*lock') }}
```

Key on a **lock-file hash**: same deps hit, changed deps rebuild.

Hosted runners for lint/docs/unit tests; self-hosted for anything that needs a GPU or accelerator.

Self-hosted runners

- Your own machine or pod, registered to the repo/org.
- `runs-on:` can be a **label set**, so a job picks a machine by capability (GPU, arch, image).
- You get special hardware; you also own maintenance and security.

Secrets & fork safety (often botched)

- Secrets live in repo/org settings, injected as `${{ secrets.NAME }}`. **Never** hard-code.
- `pull_request` **from a fork gets no secrets** (safe default), so a malicious PR cannot read them.
- `pull_request_target` runs in the **base** repo context (has secrets) but **must not** check out and run untrusted PR code. Use it only to relay a trusted action.
- Prefer **OIDC** for cloud/registry auth: short-lived tokens, nothing long-lived is stored.

Part 3 : A real public example

torch-spyre on GitHub

The `torch-spyre` repository

- Public repository: github.com/torch-spyre/torch-spyre.
- It is a **PyTorch backend** for an AI accelerator: real code, real hardware tests.
- A good example because its CI has all the hard parts: an **expensive native build**, tests that need **special hardware runners**, and a **large dynamic test matrix**.
- **16 workflows** and **8 composite actions**, all in the **public** `.github/` directory. You can read every line.

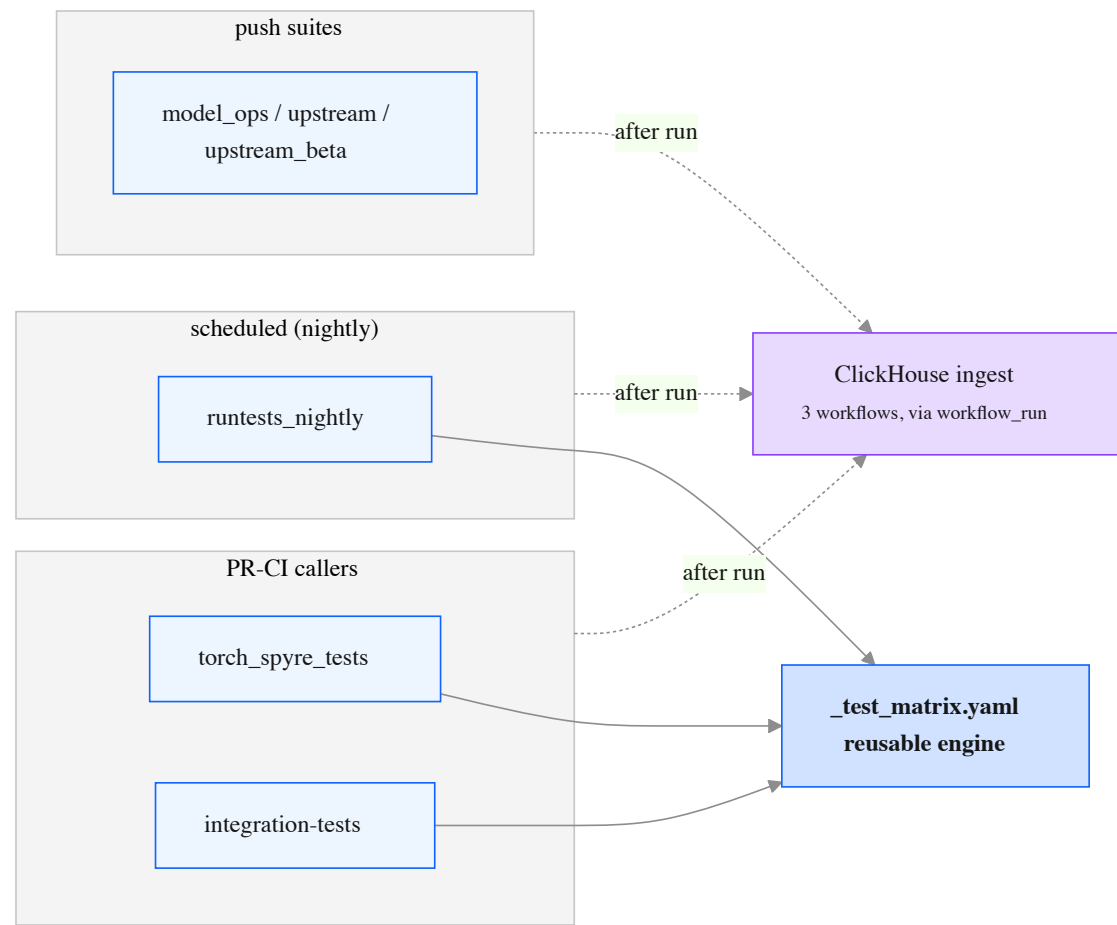
We look at the *shape*, not every line. The same shape applies to any repo with an expensive build and a big test matrix.

16 workflows, grouped into five families

Family	Workflows	Runs on
Reusable engine	<code>_test_matrix.yaml</code>	called by others
PR-CI callers	<code>torch_spyre_tests.yaml</code> , <code>integration-tests.yaml</code>	PR, merge queue, cross-repo dispatch
Scheduled (nightly)	<code>runtests_nightly.yaml</code> , <code>model_ops_tests_nightly.yaml</code>	cron <code>0 2 * * *</code>
Push test suites	<code>model_ops_tests.yaml</code> , <code>upstream_tests.yaml</code> , <code>upstream_tests_beta.yaml</code>	push to <code>main</code>
ClickHouse ingest	<code>push-to-clickhouse.yaml</code> , <code>push-hw-diagnostics-...</code> , <code>push-model-ops-logs-...</code>	after a run, via <code>workflow_run</code>
Housekeeping	<code>linters</code> , <code>docs</code> , <code>check-test-ci-coverage</code> , <code>config_integrity_checker</code> , <code>dependabot</code>	PR / push, hosted runners

One engine, many thin callers. The rest are scheduled runs, downstream ingest, and lint/docs/coverage gates.

How the families connect



The callers and scheduled runs all drive the one reusable engine; every finished run feeds the ingest workflows through `workflow_run`.

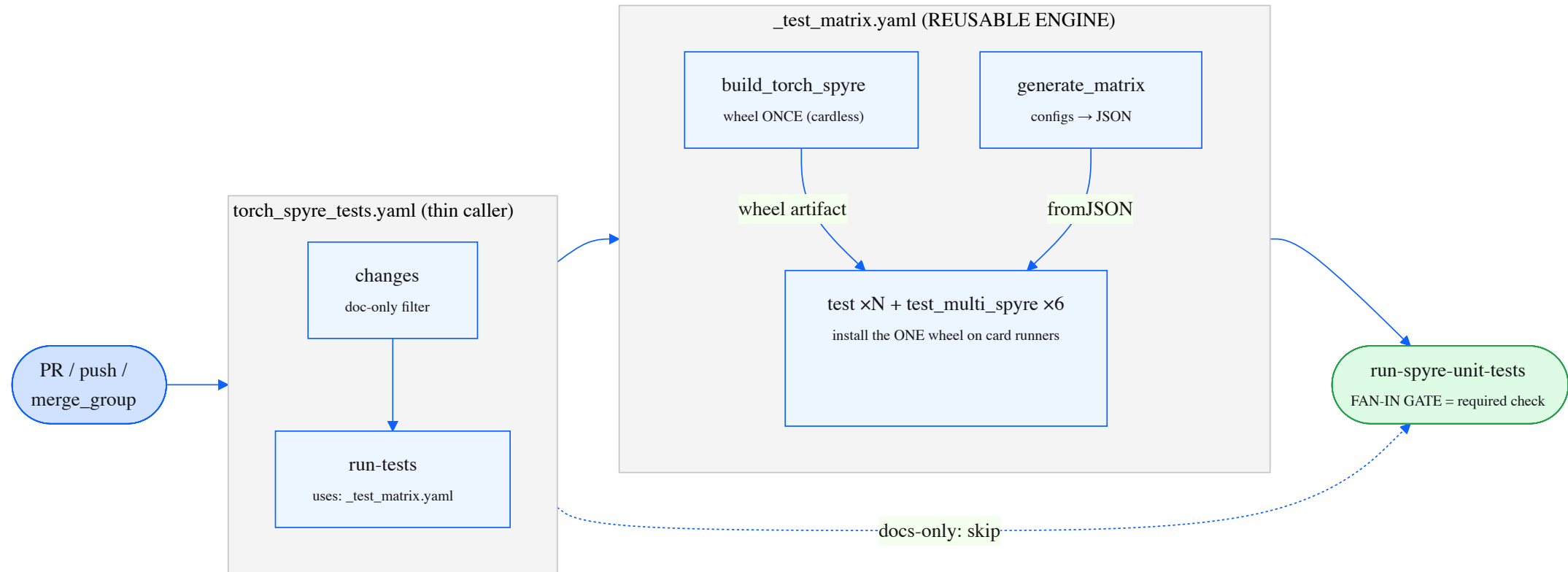
The composite-actions layer (`.github/actions/`)

The workflows above are thin because the repeated work is factored into **composite actions**:

- **build-...-wheel** : compile the C++ extension **once**, cache the compiler output, publish the wheel.
- **install-prebuilt-...** : download that one wheel and install it, **no recompile**.
- **run-test-suite** : run one config with **retry**, a **stall watchdog**, and verbose logs on the last attempt. Emits JUnit XML.
- **detect-doc-only-change** : is this PR only docs? If so, skip the hardware path.
- **gather-runner-info** : print runner diagnostics for debugging.
- **checkout-pytorch** : pin the upstream PyTorch branch to the version in `pyproject.toml`.

The build/install split is the whole trick: **one action produces the artifact, another consumes it**. That is "build once, promote the artifact" expressed as two reusable actions.

The core PR flow, end to end



`generate_matrix` emits JSON, the `test` job expands it with `fromJSON`, so new tests enter CI with **no workflow edit**. One `build_torch_spyre` produces the wheel; every matrix job just installs it.

Triggers and cadences at a glance

Trigger	Workflows	When
<code>pull_request</code> , <code>merge_group</code>	tests, linters, docs, coverage, config-check	every PR + merge queue
<code>push to main</code>	tests, model-ops, upstream, upstream-beta	on merge
<code>schedule</code> <code>0 2 * * *</code>	runtests-nightly, model-ops-nightly	nightly, 02:00 UTC
<code>workflow_dispatch</code>	tests, integration-tests, nightlies	manual / cross-repo trigger
<code>workflow_run</code> (completed)	the 3 ClickHouse ingest workflows	after a test run finishes

Only two workflows are on a timer (both nightly at 02:00 UTC). There is **no** weekly cron in this repo; everything else is event-driven. Ingest never runs on its own: it chains off a finished test run.

What happens after the tests: the ingest chain

```
tests / upstream / model-ops run  COMPLETES
```



(workflow_run: completed — decoupled, runs even if tests failed)

```
push-to-clickhouse
```

```
push-hw-diagnostics-...
```

```
push-model-ops-logs-...
```

```
parse JUnit XML
```

```
parse job LOGS
```

```
parse model logs
```

```
—> ClickHouse (test results)
```

```
—> ClickHouse (hardware failures)
```

```
—> ClickHouse (model-op results)
```

- Ingest is a **separate workflow**, not a step. It reads the finished run's **artifacts and logs**, so a crashed test run still gets its partial results recorded.
- This is a general pattern: **observability rides on workflow_run**, decoupled from the pipeline that produced the data.

The same principles, made concrete

Principle (any stack)	How this repo does it
Fail fast, fail cheap	doc-only filter + lint/coverage before the hardware matrix
Build once, promote the artifact	one wheel build → dozens of install-only test jobs
Reuse, do not copy-paste	one <code>_test_matrix.yaml</code> engine + composite actions, many thin callers
Cache deliberately	dependency + compiler (ccache) caches keyed on lock-file hashes
Make gates meaningful	one fan-in result job = the one required check
Right machine for the job	<code>runs-on</code> label set selects arch / card-count / OS / image
Decouple observability	ingest rides <code>workflow_run</code> , separate from the test pipeline

None of this is specific to accelerators. Swap "wheel" for "your build" and "hardware runner" for "GPU runner" and it is your pipeline too.

Part 4 : Jenkins

What it is and when you reach for it

What is Jenkins?

- The **veteran** CI/CD server: self-hosted, open source, predates GitHub Actions.
- A **controller** (the brain) plus **agents/nodes** (where work runs, chosen by **label**).
- **Pipeline as code**: a `Jenkinsfile` in the repo, written in Groovy.
- Enormous **plugin** ecosystem: it integrates with almost anything.

When Jenkins over GHA?

- You need **on-prem / self-hosted** control, or a private hardware fleet.
- You have **complex cross-job orchestration** (jobs triggering and waiting on other jobs).
- You already run a **large Jenkins estate** and want to build on it.

Part 5 : Jenkins basics

Declarative pipelines, agents, parallelism

Declarative Jenkinsfile anatomy

```
pipeline {  
  agent { label 'linux' } // which node runs this  
  options { timeout(time: 60, unit: 'MINUTES'); timestamps() }  
  parameters { string(name: 'REF', defaultValue: 'main') }  
  stages {  
    stage('Build') { steps { sh 'make build' } }  
    stage('Test') { steps { sh 'make test' } }  
    stage('Publish') {  
      when { branch 'main' } // a gate  
      steps { sh 'make publish' }  
    }  
  }  
  post { always { junit '**/results.xml' } } // always-run  
}
```

Same conveyor belt, different syntax. `agent` =runner · `stages` =belt · `when` =gate · `post` =always.

Agents, parallelism, reuse

- **Agents by label:** `agent { label 'linux' }, 'gpu'`, or a named machine. The controller schedules the stage onto a node that carries that label.
- **Parallelism:** build a map of branches and run them together.

```
parallel(  
  'unit':      { sh 'make unit' },  
  'integration': { sh 'make integration' }  
)
```

- **Cross-job orchestration:** `build(job: 'other-job', parameters: [...], wait: true)` triggers another job and waits for it. This is how one pipeline drives many.
- **Reuse:** a formal **Shared Library** (`vars/*.groovy`), or `load()`-ed Groovy helper files.

Scripted power, and the key lesson

- Declarative is the recommended, structured style. **Scripted** pipelines give you full Groovy when you need loops, dynamic stage generation, or complex logic.
- As pipelines grow, resist piling logic into the Groovy:

Anti-pattern	Better
Fat <code>Jenkinsfile</code> , lots of <code>sh</code> + <code>if</code>	Thin driver calls a tested script (Python/shell)
Copy-pasted stages across jobs	A Shared Library with parameters
Logic testable only by a full run	Logic in functions you can unit-test locally

Lesson: your pipeline is software. Push the logic out of the CI DSL into testable code and keep the `Jenkinsfile` a thin driver.

Part 6 : A real internal example

A shared Jenkins that orchestrates many repositories

How we use Jenkins (high-level)

We run **one shared Jenkins instance** that serves several repositories at once. It is **not** the CI that lives inside any single product repo. It sits **above** them and coordinates work **across** them. At a high level it does four things:

- **Builds container images** on a schedule, one lane per CPU architecture, in **parallel**, and pushes them to an internal registry.
- **Orchestrates cross-repo work**: it resolves a **dependency graph** across several components, dedupes, and dispatches thin per-component build jobs level by level.
- **Brokers scarce hardware**: it provisions and tears down **ephemeral test runners** on a Kubernetes cluster, so expensive accelerator capacity is used only while a test runs.
- **Runs ops jobs**: scheduled node health checks, cleanup, and cache garbage collection.

No internal code, URLs, image names, or credentials are shown here. This is the design, not the source.

Why a shared Jenkins, and one design lesson

- **Why Jenkins for this:** it holds credentials to private infrastructure, it schedules onto a private hardware fleet by label, and it coordinates **many jobs** driving each other. That is exactly Jenkins' strength.
- **The orchestration logic lives in a tested language** (a small library), not in the **Jenkinsfile**. The Jenkins layer stays a thin driver that calls it. This mirrors the "your pipeline is software" lesson from Part 5.
- **The runner fleet is declarative** (GitOps): its configuration is version-controlled and rolled out through review, not hand-edited on the cluster.

Takeaway: use Jenkins where it is strongest (private orchestration + hardware brokering) and keep the intelligence in code you can test.

Part 7 : Jenkins → GHA → Jenkins

Integrating both for complex workflows

The problem this solves

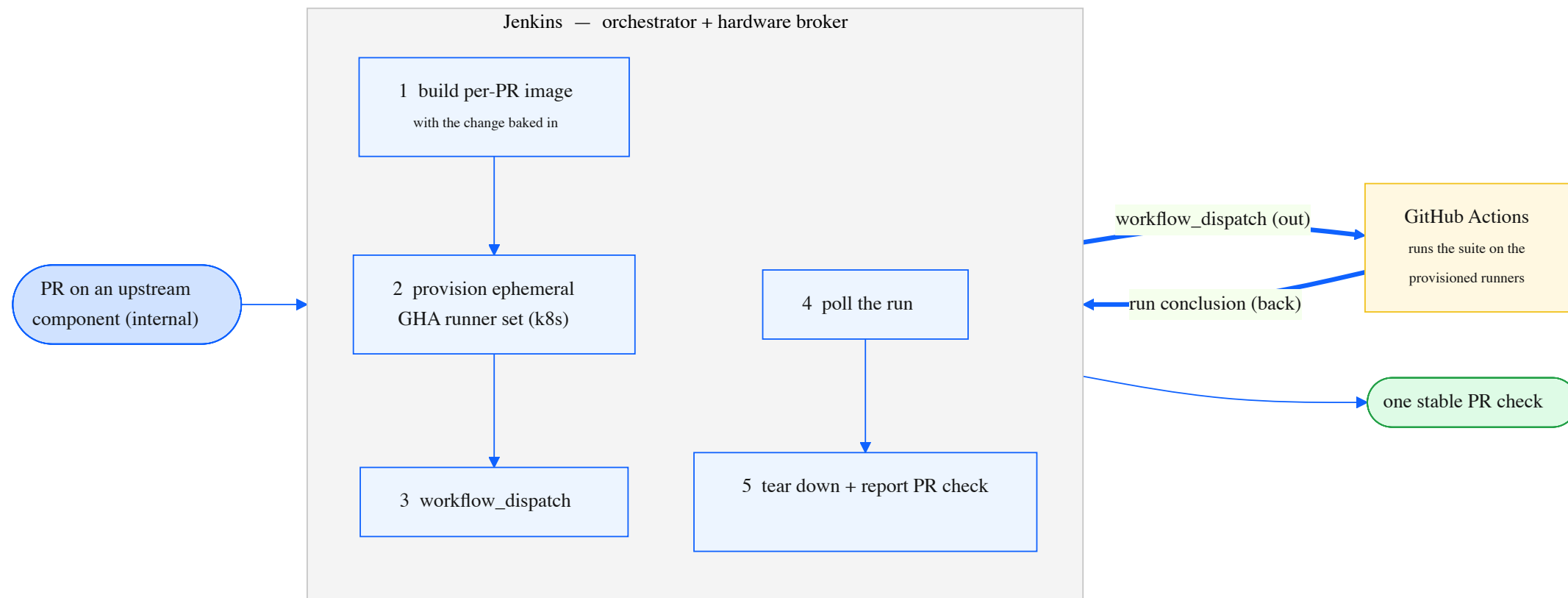
Several repositories **depend on each other**. A change in a low-level component can break a downstream one.

Question: when someone opens a PR on an upstream component, how do you run the *downstream* test suite, on real hardware, and report one clear pass/fail back on that PR?

Answer: let Jenkins and GitHub Actions each do what they are best at, and connect them at a single, stable seam.

- **Jenkins** = orchestrator + hardware broker + image builder.
- **GitHub Actions** = the in-repo test executor.

The round trip



- Out from Jenkins: a **workflow_dispatch** into the public repo's workflow.
- Back to Jenkins: the **run conclusion**, turned into a **PR check** on the internal PR.

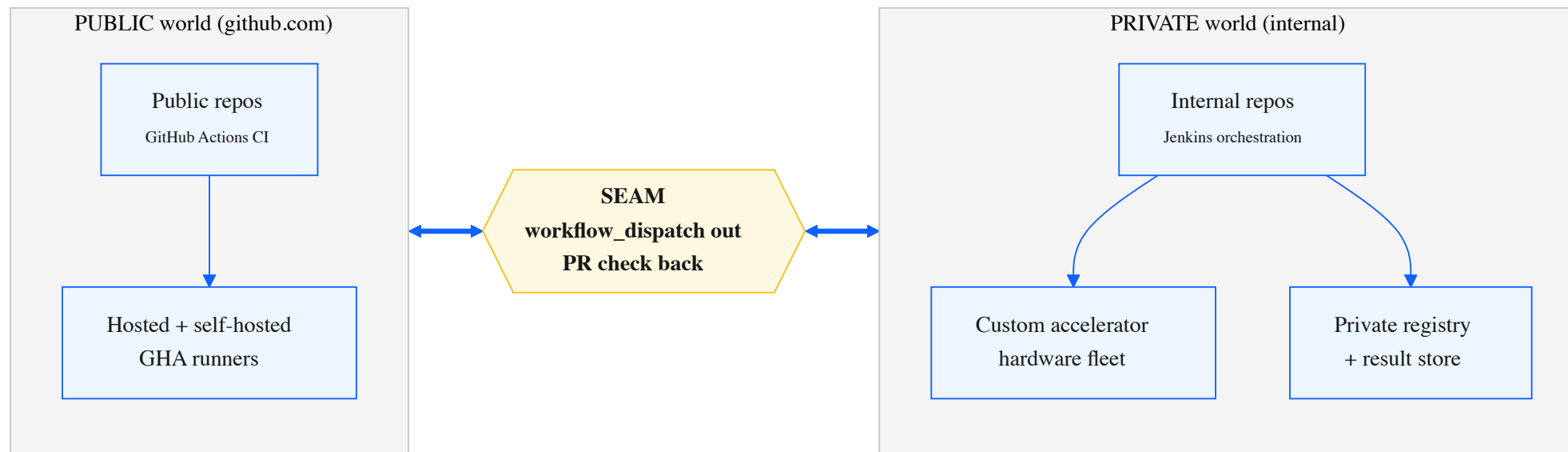
Why this shape (the takeaways)

- **workflow_dispatch** is the seam. It is a single, documented entry point: Jenkins triggers a named public workflow with inputs, and treats the run result as the contract. Neither side needs to know the other's internals.
- **Per-PR image with the change baked in** → the test runs against the *exact* candidate build, not a patched-at-runtime approximation. (Reproducibility.)
- **Ephemeral runners** → created for the test, destroyed after → clean environment every time, and scarce hardware is not held idle. (Cost + isolation.)
- **One stable check name** back on the PR → branch protection and pollers do not care how the internals change. (Stable contract.)

Every piece here is a general principle from the earlier parts, wired across two tools.

Why this is hard: two worlds, one pipeline

Our stack lives in **two worlds** at once, and a single change has to cross the boundary between them.



Public repos + GitHub Actions on one side; internal repos, custom accelerator hardware, and a private registry on the other. The `workflow_dispatch` seam is the *only* sanctioned crossing point.

The challenges, and how we solved them

Challenge	How we solved it
Custom hardware not on any hosted runner	Self-hosted runners provisioned on our own cluster; jobs select hardware by label set
Code split across internal + external repos	Each repo keeps its own in-repo CI; a shared Jenkins orchestrates across them by a dependency graph
Different tech per context (GHA public, Jenkins internal)	Use each where it is strongest, joined at one documented seam (workflow_dispatch out, PR check back)
Code + data privacy (public repo, private infra)	Nothing internal leaks into public workflows; secrets stay in the internal side; fork PRs get no secrets
Scarce, expensive hardware held idle	Ephemeral runner sets: created for one test, torn down after, so cards are used only while a test runs
Visibility + monitoring across both worlds	Results ingested to a central store via decoupled workflow_run jobs; one stable PR check is the contract

The theme: let each tool do what it is best at, and connect them at a single narrow seam you can secure, monitor, and reason about.

GitHub Actions vs Jenkins : pick per job

	GitHub Actions	Jenkins
Setup	Zero: built into GitHub	Self-host controller + agents
Config	YAML workflows	Groovy Jenkinsfile
Runners	Hosted + self-hosted (incl. ephemeral pods)	Your agents, by label
Reuse	Composite actions, reusable workflows	Shared libraries / load()
Best at	In-repo CI, cloud-native, quick start	On-prem control, complex orchestration, private fleets

They are complementary. We use **GHA** for in-repo CI and **Jenkins** as the cross-repo hardware-brokering orchestrator, joined at the `workflow_dispatch` seam. Pick per job, not per religion.

Part 8 : References

References

GitHub Actions

- Docs: docs.github.com/actions
- Reusable workflows: docs.github.com/actions/using-workflows/reusing-workflows
- Self-hosted runners: docs.github.com/actions/hosting-your-own-runners
- Security hardening: docs.github.com/actions/security-guides/security-hardening-for-github-actions
- Public example repo: github.com/torch-spyre/torch-spyre (see [.github/workflows/](https://github.com/torch-spyre/torch-spyre/.github/workflows/) and [.github/actions/](https://github.com/torch-spyre/torch-spyre/.github/actions/))

Jenkins

- Docs: jenkins.io/doc
- Pipeline syntax: jenkins.io/doc/book/pipeline/syntax
- Shared libraries: jenkins.io/doc/book/pipeline/shared-libraries

Concepts

- Continuous Delivery, Humble & Farley

Thank you!

Questions?

Harikrishnan Balagopal · IBM Research

GitHub Actions and Jenkins are two engines for the same conveyor belt. Learn the parts of each, then connect them at a clean seam for the hard workflows.